

$$a) T(n) = 16T\left(\frac{n}{4}\right) + n!$$

$$a=16$$

$$b=4$$

$$f(n)=n!$$

$$\Omega(n \log_4^{16} + \epsilon)$$

$$n \log_b^a = n \log_4^{16} = n^2$$

since $n! > n^2 \rightarrow$ Case 3 applies

$$T(n) = \Theta(f(n)) = \Theta(n!)$$

$$b) T(n) = \sqrt{2}T\left(\frac{n}{4}\right) + \log n$$

$$a=2^{1/2}$$

$$b=4$$

$$f(n)=\log n$$

$$n \log_b^a = n \log_4^{\sqrt{2}} = n^{0.25}$$

$\log n$ is smaller $\rightarrow$ meaning $\log n = n^{0.25-\epsilon} \rightarrow$ First rule

$$T(n) = \Theta(n \log_b^a) = \Theta(n \log_4^{\sqrt{2}}) = \Theta(n^{0.25})$$

$$c) T(n) = 8T\left(\frac{n}{2}\right) + 4n^3$$

$$a=8$$

$$b=2$$

$$f(n)=4n^3$$

$$c=3$$

$$n \log_b^a = n \log_2^8 = n^3$$

$4n^3$ is asymptotically same as n^3

$$\log_2^8 = 3 \rightarrow$$

Case 2

$$\rightarrow \Theta(n \log_b^a \cdot \log n)$$

$$T(n) = \Theta(n^3 \cdot \log n)$$

$$d) T(n) = 64T\left(\frac{n}{8}\right) - n^2 \log n$$

$f(n)$, which is the combination of time cannot be negative

Theorem does not apply.

$$e) T(n) = 3T\left(\frac{n}{3}\right) + \sqrt{n}$$

$$a = 3$$

$$b = 3$$

$$f(n) = \sqrt{n}$$

$$c = 1/2$$

$$\log_b^a = \log_3^3 = 1$$

$$1 > c$$

$$1 > 1/2 \Rightarrow$$

$$\text{Case 1} \rightarrow T(n) = \Theta(n^{\log_b^a}) = \Theta(n^1) = \underline{\underline{\Theta(n)}}$$

$$f) T(n) = 2^n \cdot T\left(\frac{n}{2}\right) - n^n$$

$$a = 2^n$$

$$b = 2$$

$$f(n) = -n^n$$

$$\left\{ \begin{array}{l} a \text{ is not constant} \\ f(n) \text{ is not a polynomial} \end{array} \right. \Rightarrow \text{Theorem does not apply.}$$

$$g) T(n) = 3T\left(\frac{n}{3}\right) + \frac{n}{\log n}$$

$$a = 3$$

$$b = 3$$

$$f(n) = \frac{n}{\log n}$$

$$\left\{ \begin{array}{l} f(n) \text{ is not a polynomial} \end{array} \right. \Rightarrow \text{Theorem does not apply.}$$

Q2

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

a = subproblem

$\frac{n}{b} \rightarrow$ size of the sub-problem

a) $T(n) = 9T\left(\frac{n}{3}\right) + n^2 \rightarrow$ quadratic time

$\uparrow$
 9 sub-problem $\rightarrow$ one-third of its size

$$a = 9$$

$$b = 3$$

$$f(n) = n^2$$

$$c = 2$$

$$\log_b^a = \log_3^9 = 2$$

$$\text{Since } \log_b^a = c \Rightarrow 2 = 2$$

second case $\rightarrow T(n) = \theta(n^{\log_b^a} \cdot \log n)$
 $= \theta(n^2 \log n)$

b) $T(n) = 8T\left(\frac{n}{2}\right) + n^3$

$$a = 8$$

$$b = 2$$

$$f(n) = n^3$$

$$c = 3$$

$$\log_b^a = \log_2^8 = 3$$

$$\text{Since } \log_b^a = c \Rightarrow 3 = 3$$

second case

$$T(n) = \theta(n^{\log_b^a} \cdot \log n)$$

$$= \theta(n^3 \cdot \log n)$$

c) $T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$

$$a = 2$$

$$b = 4$$

$$f(n) = \sqrt{n}$$

$$c = 1/2$$

$$\log_b^a = \log_4^2 = 1/2$$

$$\text{Since } \log_b^a = c \Rightarrow \frac{1}{2} = \frac{1}{2}$$

second case

$$T(n) = \theta(n^{\log_b^a} \cdot \log n)$$

$$= \theta(\sqrt{n} \cdot \log n)$$

Running times of algorithm

$$O(\sqrt{n} \cdot \log n) < O(n^2 \cdot \log n) < O(n^3 \cdot \log n)$$

$\downarrow$
c

$\downarrow$
a

$\downarrow$
b

c is the fastest
b is the slower.

$\Rightarrow$ I would prefer Algorithm c

Q3

Merge Sort

Best case : $\Omega(n \cdot \log n)$
 Average case : $\Theta(n \cdot \log n)$
 Worst case : $O(n \cdot \log n)$

Quick sort

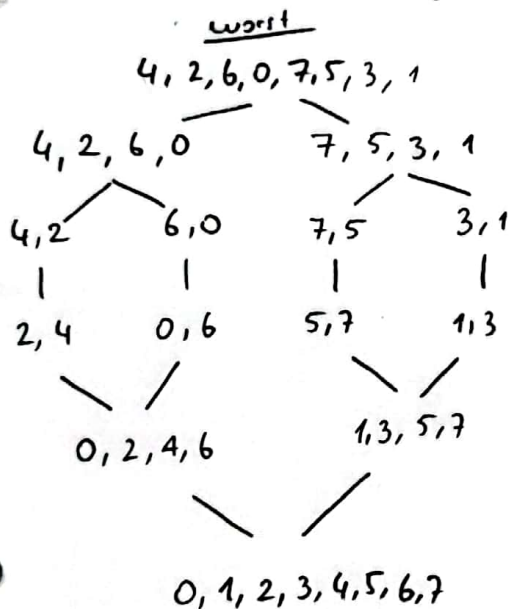
Best case : $\Omega(n \cdot \log n)$
 Average case : $\Theta(n \cdot \log n)$
 Worst case : $O(n^2)$

1) Mergesort maximum comparison: Every pair should be compared and they should be in wrong order.

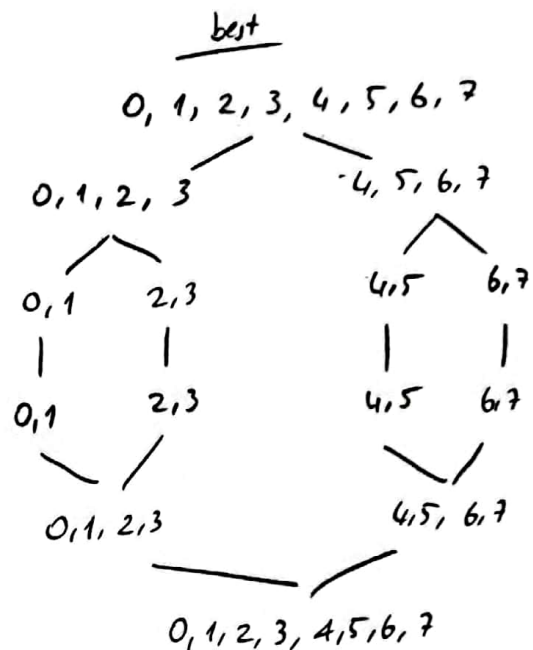
Mergesort minimum comparison: Every pair is also compared, but they are already sorted.

PS: Since comparison must occur at all costs, best case is the same as worst case.

But I will still give examples.



↑
At every step, index changes happen.



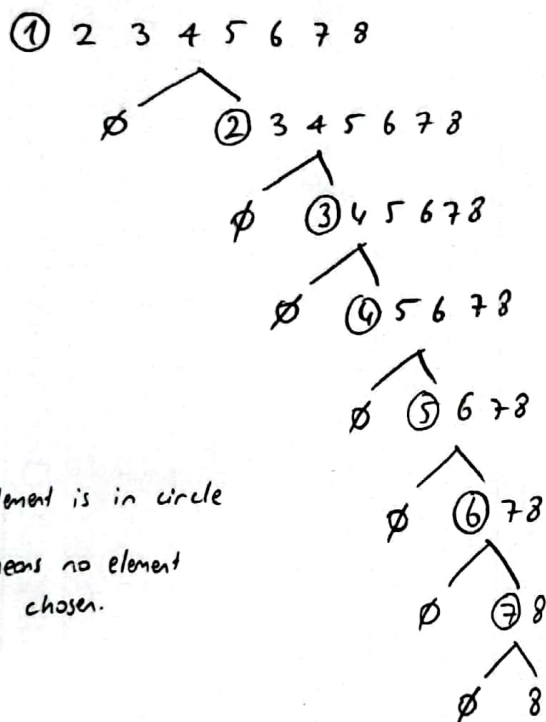
↑
No index changes happen in no steps.

Number of comparisons is still the same. Best case and worst case has no difference.

- b) Quick sort is worst when the list is already sorted in either ascending or descending order. Because there is nothing to compare to in one side.

Quick sort is best when median element is pivot, if possible. If not, the more random the list, the quicker the sorting. Remember, best case and the average case is same for quick sort. $\Theta(n \cdot \log n)$

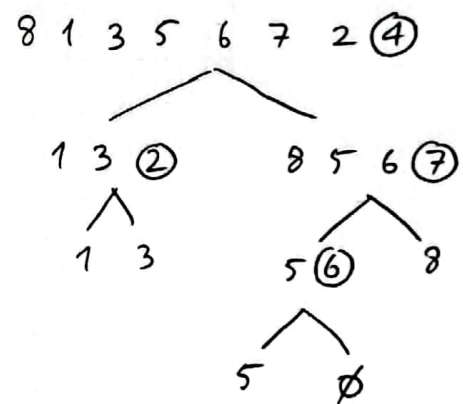
worst case (more comparison)



pivot element is in circle

$\emptyset \rightarrow$ means no element is chosen.

best case (less comparison)



it would be best, if # of elements in the list is an odd number, because it evenly distributes to both sides. But question asked for 8 numbers.

Q4

This algorithm is a binary search algorithm, that only searches the number "0".
It does not work for other numbers.

First, we check if $A[n/2]$ is 0. If $0 > A[n/2]$ delete the lower half.

If $0 < A[n/2]$, delete the upper half. This means at each iteration, problem size reduces by half.

↳ Thus the recurrence relation $\rightarrow T(n) = T(\frac{n}{2}) + 1$

$$1^{\text{st}} \text{ step} \rightarrow T(n) = T(\frac{n}{2}) + 1$$

$$2^{\text{nd}} \text{ step} \rightarrow T(\frac{n}{2}) = T(\frac{n}{4}) + 1 + 1$$

$$3^{\text{rd}} \text{ step} \rightarrow T(\frac{n}{4}) = T(\frac{n}{8}) + 1 + 1 + 1$$

⋮

$$k^{\text{th}} \text{ step} \rightarrow T(\frac{n}{2^{k-1}}) = T(\frac{n}{2^k}) + 1 \cdot k \rightarrow \text{final step}$$

$\Rightarrow \frac{n}{2^k} = 1 \rightarrow$ How many times do we need to divide by 2, until only one element left.

$\Rightarrow n = 2^k \rightarrow$ Take the log on both sides

$\Rightarrow \log n = k \rightarrow$ Put k in the final step

$$T(n) = T\left(\frac{n}{2^{\log n}}\right) + 1 \cdot \log n$$

$$T(n) = T(1) + \log n$$

we know that $T(1)$ is 1 because it is a base condition; since we are left with only 1 element to be searched.

⇓

$$T(n) = 1 + \log n$$

$$\text{Thus ; } T(n) = \Theta(\log n)$$

Note: The question can be solved by Master Theorem

$$a = 1$$

$$b = 2$$

$$f(n) = n^0$$

$$c = 0$$

$$\log_b^a = \log_2^1 = 0$$

$$\text{if } \log_b^a = c \Rightarrow \text{then}$$

$$0 = 0 \checkmark$$

$$T(n) = \Theta(n^c \log n)$$

$$= \Theta(\log n)$$

Q5

```

function Quicksort(arr1, arr2, left, right)
  if (left < right) do:
    var pivot = Partition(arr1, left, right, arr2[right])
    Partition(arr2, left, right, arr1[pivot])
    Quicksort(arr1, arr2, left, pivot-1)
    Quicksort(arr1, arr2, pivot+1, right)
  end if
end function

```

```

function Partition(arr, left, right, pivot)
  var i = left
  var j = left
  while(j < right) do:
    if(arr[j] < pivot) do:
      swap(arr[i], arr[j])
      i = i+1
    end if
    else if (arr[j] == pivot) do:
      swap(arr[j], arr[i])
      j = j+1
    end elseif
    j = j+1
  end while
  swap(arr[i], arr[right])
  return i;
end function

```

Recurrence Relation

$$T(n) = 2T\left(\frac{n}{2}\right) + n \rightarrow \text{time}$$

↳ 2 subproblems
↳ split in half

By Master Theorem

$$a=2$$

$$b=2$$

$$f(n) = n^1$$

$$c=1$$

$$\log_b^a = \log_2^2 = 1$$

$$\text{if } \log_b^a = c \Rightarrow \text{case 2 } \checkmark$$

$$T(n) = (n^{\log_b^a} \cdot \log n)$$

$$= (n^1 \cdot \log n)$$

$$= \underline{(n \cdot \log n)}$$